

Implementando as árvores de decisão

Olá!

Neste vídeo, iremos aprender a utilizar as árvores de decisão no SK-learn.

Para isso, iremos aplicar tanto a versão de classificação quanto a versão de regressão da árvore de decisão.

Então, iremos iniciar importando as bibliotecas que iremos utilizar.

Em uma célula vazia, iremos utilizar:

```
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor. "Tree", de árvore.
```

Dessa maneira, na mesma linha, estamos importando tanto a versão da árvore de decisão para classificação, quanto a versão para regressão.

Iremos, também, na linha seguinte, importar os data sets que utilizaremos, então:

```
from sklearn.datasets import load_boston, load_iris. O dataset de "boston", para regressão e o dataset de "iris", para classificação.
```

E iremos importar, também, o "train_test_split" que seria a função para efetuar a divisão em "train" e "test".

Então, teremos:

```
from sklearn.model_selection import train_test_split.
```

Iremos executar esta célula com os imports e iremos carregar os conjuntos de dados.

Primeiramente, eu irei aplicar a árvore de decisão no conjunto de classificação utilizaremos, primeiro, a árvore de classificação que seria o conjunto de dados "iris" e, depois, iremos aplicar árvore de decisão na regressão que seria no conjunto de dados de "boston".

Iremos fazer o seguinte: na próxima linha, iremos aplicar:

```
dt = DecisionTreeClassifier(). Utilizaremos, aqui, os parênteses, abre, "(", e fecha parênteses, ")" para indicar que estamos instanciando esta árvore.
```

Iremos executar esta linha.

E na próxima linha, iremos carregar e preparar os dados de "train" e "test".

Então, iremos utilizar, na próxima linha:

```
X,y = load_iris(return_X_y=True).
```

Dessa maneira, estamos carregando conjunto de dados de "iris" nas variáveis "X" e "Y" onde "X" serão os atributos, e "Y" os rótulos.

Na próxima linha iremos efetuar a divisão em "train" e "test":

```
X_train, X_test, y_train, y_test = train_test_split(X,y).
```

Ao executar esta linha, teremos a divisão em "train" e "test".

Ao aplicar os dados na árvore de decisão iremos construir a árvore, então, iremos aplicar: `dt.fit(X_train,y_train)`.

Dessa maneira, árvore foi treinada de forma simples como podemos perceber.

E iremos aplicar o "score" para saber qual é o resultado do teste da árvore. Então, usaremos: `dt.score(X_test,y_test)`.

Ao aplicar esse resultado, temos uma precisão de 92% de acerto. Tivemos um acerto de 92% no conjunto de dados "iris" que é um conjunto de dados de classificação e iremos aplicar, agora, na regressão passos que são bem parecidos.

Então, usaremos: `dt = DecisionTreeRegressor()`.

Desse modo, instanciamos o modelo de árvore de decisão para regressão.

Prepararemos o conjunto de dados de maneira bem similar à anterior: `X,y = load_boston(return_X_y=True)`

Dessa forma, carregamos os dados nas variáveis "X" e "Y". A seguir, iremos utilizar: "`dt.fit`" para carregar os dados, porém, antes, iremos utilizar a divisão em "train" e "test".

Iremos utilizar:

`X_train, X_test, y_train, y_test = train_test_split(X,y)`.

Pronto, agora, podemos aplicar o "`fit`" para treinar a árvore de decisão com esse conjunto de dados.

Então, teremos: `dt.fit(X_train,y_train)`.

Dessa maneira, temos o regressor sendo treinado. Ao executar esta célula, na linha seguinte, iremos utilizar: `dt.score(X_test,y_test)`.

E conseguimos um acerto de 71% para esse conjunto de dados.

O primeiro ponto que precisamos verificar é que a árvore de decisão por natureza já possui um alto índice de overfitting porque, pela sua natureza, ela tenta sempre carregar todo o conjunto de dados e tenta separá-lo perfeitamente.

No que isso implica?

Uma vez que tentamos separar o conjunto de "train" perfeitamente temos o overfitting, já que se existirem dados outliers dados que não deveriam pertencer à normalidade nós teríamos uma divisão desses dados os tratando como se eles fossem, de fato, dados válidos e temos o caso de overfitting.

O overfitting se evidencia quando as árvores conseguem separar os dados perfeitamente.

Mas como assim?

Se o objetivo da árvore de decisão é, exatamente separar perfeitamente os dados, por que isso seria ruim?

Podemos pensar da seguinte maneira:

O conjunto de dados de "train" mostra algumas das características dos dados logo, iremos tentar extrair os padrões desse conjunto de dados para que a árvore de decisão consiga capturar esses padrões e classificar novos elementos, de acordo com os padrões encontrados.

Porém, imagine o seguinte: você está querendo estudar para uma prova.

Se você apenas estuda as mesmas questões ou uma mesma questão sobre um determinado assunto você vai aprender apenas aquela questão, você não vai aprender a generalizar o conhecimento sobre o assunto.

Usando essa analogia, podemos trazer esse conhecimento para as árvores. Se as árvores aprendem perfeitamente a separar um conjunto de dados a classificar um conjunto de dados elas podem classificar esse conjunto de dados tão perfeitamente que elas não conseguirão generalizar esse aprendizado para outros conjuntos.

Logo, é melhor que possamos trabalhar com uma árvore que não classifique tão perfeitamente esse conjunto de dados, mas que consiga generalizar para os demais. Ao invés de aprender perfeitamente um conjunto de dados a ideia é atender os conceitos daquele conjunto de dados. Então, para isso, nós podemos utilizar uma quantidade menor de altura da árvore para que possamos generalizar o conhecimento.

Uma vez que uma altura menor da árvore irá garantir que nós não continuaremos fazendo as divisões dos dados perfeitamente nós iremos fazer as divisões até um certo ponto de modo que os conceitos gerais sejam apreendidos e o restante dos conceitos não seja perfeitamente apreendidos uma vez que não se faz necessário que apreendamos perfeitamente o conjunto de dados de entrada, mas sim que possamos pegar uma ideia do que esses conjuntos de dados significam.

Então, temos que, de alguma maneira diminuir a quantidade de overfitting neste conjunto de dados aplicando alguma técnica para podar a árvore, ou seja, evitar que ela tenha tantos nós. Então, a partir deste momento, irei retreinar a árvore utilizando um limitador da quantidade de regras que serão feitas. Ao limitar essa quantidade de regras nós teremos uma diminuição da quantidade de nós puros.

A divisão dos dados pode não ficar pura e aí, por exemplo, no caso da classe iremos utilizar a maioria das classes desse nó supondo que chegamos em um nó.

Em uma situação em que temos cinco salmões e um sea bass em um nó e não continuaremos separando esses conjuntos de dados, mas iremos escolher a classe como a classe dos salmões. Qualquer elemento nesse nível será classificado como salmão.

Mesmo que o nó esteja puro, iremos escolher a maioria.

No caso de um conjunto de dados de regressão iremos escolher a média dos valores que estão ali.

Então, vamos reaplicar o modelo dessa vez, executando uma poda na árvore.

Iremos utilizar novamente, na próxima linha: `dt = DecisionTreeClassifier(max_depth=)`. Esse parâmetro "depth" seria a profundidade máxima.

E, aqui, podemos dizer a altura máxima da árvore que iremos ter.

Quanto menor essa altura, menos divisões teremos e se ela for pequena demais podemos ter um underfitting.

Se essa árvore tiver nós demais ou uma altura muito grande podemos ter overfitting. Então, cabe muito do cuidado e testes para verificar qual é o tamanho ideal da árvore que iremos utilizar.

Neste conjunto, por exemplo, experimentaremos altura máxima "5". Ao executarmos

esta altura máxima de 5 podemos aplicar novamente o conjunto de dados:

`X,y = load_iris(return_X_y=True)`. E iremos efetuar a divisão de "train" e "test" desses dados: `X_train, X_test, y_train, y_test = train_test_split(X,y)`. Agora, podemos retreinar a árvore com os dados de "train" e aplicar "score", para verificar os dados de "test". Iremos utilizar: `dt.fit(X_train,y_train)`.

E, assim, construímos a árvore de decisão para classificação com altura máxima de 5.

Ao efetuarmos: `dt.score(X_test,y_test)`, temos que o acerto é de 92% ainda similar ao que tínhamos antes.

Ao diminuirmos a altura máxima da árvore para "3" e reexecutarmos o "test" temos um aumento no "score" de "test" para 94,73% ou seja, uma árvore menor, mais simples, mas que deu origem a resultados melhores.

Então, vale a pena continuar executando esses testes de profundidade para encontrar o melhor valor possível para o "score" da árvore uma vez que ela é altamente sensível à profundidade.

Encerramos este vídeo tendo construído uma árvore de decisão e aplicado a técnica de poda, para evitar overfitting.

No próximo vídeo, iremos aprender outra peculiaridade da árvore de decisão a respeito das features utilizadas para construir essa árvore aos atributos, às colunas de dados.

Até lá!